Threat Model Report

Source: C:/Users/Administrator/Desktop/flask_todo_app/flask_todo_app/app/THREATMODEL.md

Threat Model: Flask To-Do Application

Scope (Code-Aware)

This threat model is based on the current code in:
* app/routes/auth.py (/register, /login, /logout)
* app/routes/tasks.py (/, /tasks/create, /tasks/<int:task_id>/edit, /tasks/<int:task_id>/delete)
* app/models/user.py and app/models/task.py
* app/__init__.py and config.py

Data is stored in SQLite (todo.db) with Task.user_id as the ownership boundary.

STRIDE Threat Model

```
| STRIDE category | Threat description | Example in this To-Do app | Mitigation status in code |
|---|---|---|---|
| **S ? Spoofing** | Attacker acts as another user by stealing credentials/session. | POST to /login with stole
then access / to read tasks. | Passwords are hashed with generate_password_hash() and verified by check_passwor
app/models/user.py; authenticated routes use @login_required. Gap: no lockout/rate limit on login. |
| **T ? Tampering** | User alters task data they do not own. | Change task_id in /tasks/<id>/edit or /tasks/<id
Fixed in app/routes/tasks.py: _get_user_task_or_404() uses Task.query.filter_by(id=task_id,
user_id=current_user.id).first_or_404(). |
| **R ? Repudiation** | User denies making a change; no forensic trail. | User claims they did not delete a tas
/tasks/<id>/delete. | Gap: app uses flash messages only; no persistent audit log (user id, IP, timestamp, route
| **I ? Information Disclosure** | Unauthorized data exposure between users or via unsafe runtime config. | Vie
tasks by id enumeration; stack traces if run with debug on. | Per-user list query exists on / (filter_by(user_i
and object-level check is in place. Gap: run.py currently starts Flask with debug=True. |
| **D ? Denial of Service** | Resource exhaustion blocks legitimate use. | Repeated POST flooding on /login or
saturates app/DB. | Gap: no request throttling, CAPTCHA, or reverse-proxy limit in code. |
| **E ? Elevation of Privilege** | Regular user performs actions outside their scope. | Normal user attempts ta
records owned by another user. | Mitigated for task ownership via user-scoped query. No admin role exists in cu
role-escalation surface. |
```

Data Flow Diagram (DFD)

```
 [Browser]
   | 1) POST /register (username,password)
   | 2) POST /login (username,password) -> session cookie
   | 3) POST /tasks/create, /tasks/<id>/edit, /tasks/<id>/delete (with csrf_token)
   v
 [Flask app: auth_bp + tasks_bp]
   |-- Flask-Login: current_user, @login_required
   |-- Flask-WTF CSRFProtect: validates csrf_token on unsafe methods
   |-- after_request headers:
   |      X-Frame-Options: DENY
   |      Content-Security-Policy: frame-ancestors 'none'
   v
 [SQLAlchemy models]
   |-- User(id, username, password_hash)
```

```
   |-- Task(id, title, completed, created_at, user_id)
   v
 [SQLite todo.db]
```

Trust boundaries

* Browser to Flask app: untrusted input boundary.
* Flask app to SQLite: trusted app/data boundary.
* Session cookie in browser: client-controlled storage boundary.

Simple Attack Tree: Unauthorized Task Access

```
Goal: Read/modify another user's task
|
+-- [OR] IDOR on task endpoints
|    +-- Authenticate as any user
|    +-- Send /tasks/2/edit or /tasks/2/delete for foreign record
|    +-- Blocked now by filter (id + current_user.id)
|
+-- [OR] CSRF against authenticated victim
|    +-- Victim logged in
|    +-- Victim visits attacker page
|    +-- Hidden form posts to /tasks/<id>/delete
|    +-- Blocked now by missing/invalid csrf_token
|
+-- [OR] Clickjacking-assisted action
     +-- App framed by attacker page
     +-- Victim trick-clicks delete/logout
     +-- Blocked now by X-Frame-Options + CSP frame-ancestors
```

Current Implemented Mitigations (From Code)

* IDOR: object lookup constrained by both task_id and current_user.id.
* CSRF: csrf.init_app(app) plus hidden csrf_token fields in auth/task forms.
* Logout hardening: /logout changed to POST + CSRF token in navbar form.
* Session hardening: SESSION_COOKIE_SAMESITE = "Lax" in config.py.
* Clickjacking: X-Frame-Options: DENY and Content-Security-Policy: frame-ancestors 'none' in app.after_request.

Residual Gaps To Track

* run.py uses debug=True; set to False in production execution path.
* No login rate limit on /login.
* No audit log for task CRUD and auth events.

CVSS v3.1 Risk Ratings

```
| Vulnerability | Base score | Severity |
|---|---:|---|
| IDOR (unauthorized task access) | 6.5 | Medium |
| CSRF (unauthorized form submission) | 8.8 | High |
| Clickjacking | 5.4 | Medium |
```